```
Output:
Array
(
    [0] => Array
        (
            [registration] => AAA 1A
            [model] => Ford Prefect
        )

)
```

`$advertised` and `$outForRepair` contain collections of data for each car (represented here as sub-arrays, they could just as easily have been objects). We have two cars advertised, one out for repair, and the one out for repair happens to also be currently on sale. Our code aims to discover which cars need to be removed from sale by finding the intersect (the idea being that we can then send the intersect results elsewhere to update the sales database).

The only reliable way to compare the different array structures is via the common `'registration'` key, so we have an anonymous function as the third argument to `array_uintersect()`, which performs a comparison between two elements' `'registration'` value (see Chapter 3, "Functions," if you're not sure what an anonymous function is). The callback function is called every time `array_uintersect` compares elements between its own input arrays; each element being compared comprises the arguments to the callback. Elements from the first array `$advertised` will always passed to the callback as the first argument, `$outForRepair` as the second.

An important point to note is the type of comparison we are performing. We're using the spaceship operator `<=>`, which compares two values and returns `-1`, `0`, or `1` depending on whether the first operand is less than, equal to, or greater than the second. The callback function must return an integer less than, equal to, or greater than zero. The reason why the callback has to return in this way, and not just return a boolean to indicate a match, is because it isn't just used for computing the intersect. (Note that when the spaceship operator is used with string operands, it is alphabetical precedence that is tested.)

Remember that `array_diff()` and `array_intersect()` will sort input arrays before comparison? With `array_uintersect()` *the callback is also used for sorting*. This is a subtle point and isn't made clear in the official PHP documentation. If we had simply